

NS-BRKGA para o Multi-Objective Time- and Machine-Dependent Scheduling Problem

Luis H. P. Mendes Fábio L. Usberti Mário C. San Felice Carlos E. de Andrade

Instituto de Computação — Universidade Estadual de Campinas (UNICAMP)
Departamento de Computação — Universidade Federal de São Carlos (UFSCar)
AT&T Labs Research

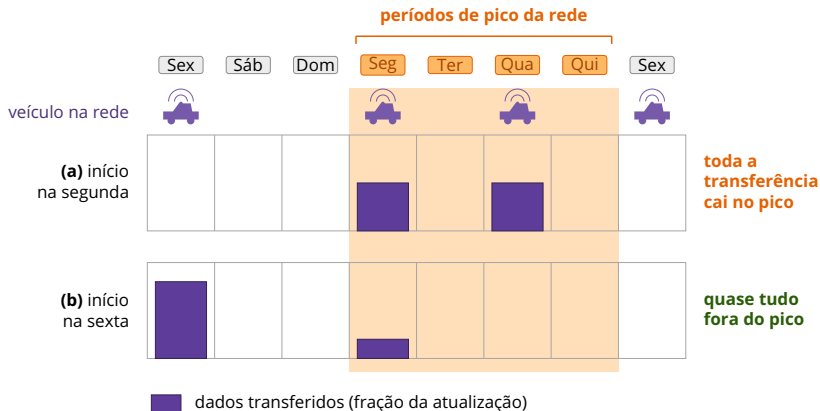
`luis.mendes@ic.unicamp.br`

LVIII Simpósio Brasileiro de Pesquisa Operacional — SBPO 2026

Em FOTA, o que se decide é *quando* começar

Um veículo conectado:

- fica na rede **poucas horas por dia;**
- sob **setores diferentes** a cada dia;
- iniciada a transferência, a rede **não a interrompe.**



Cenário de FOTA para carros conectados — Andrade et al. (2019)

Do FOTA ao TMDSP

O FOTA é modelado pelo *Time- and Machine-Dependent Scheduling Problem* (TMDSP), de Andrade et al. (2019):

Dados

- J — **tarefas**: veículos, com demanda d_j .
- M — **máquinas**: setores da rede.
- T — **períodos** do horizonte.

Regras

Do FOTA ao TMDSP

O FOTA é modelado pelo *Time- and Machine-Dependent Scheduling Problem* (TMDSP), de Andrade et al. (2019):

Dados

- J — **tarefas**: veículos, com demanda d_j .
- M — **máquinas**: setores da rede.
- T — **períodos** do horizonte.
- c_{mt}, η_{mt} — capacidade e taxa da máquina, **variáveis no tempo**.

Regras

Do FOTA ao TMDSP

O FOTA é modelado pelo *Time- and Machine-Dependent Scheduling Problem* (TMDSP), de Andrade et al. (2019):

Dados

- J — **tarefas**: veículos, com demanda d_j .
- M — **máquinas**: setores da rede.
- T — **períodos** do horizonte.
- c_{mt}, η_{mt} — capacidade e taxa da máquina, **variáveis no tempo**.
- q_{jmt} — capacidade que j consome em m no período t ; **só existe** se o veículo estiver sob aquele setor ali.

Regras

Do FOTA ao TMDSP

O FOTA é modelado pelo *Time- and Machine-Dependent Scheduling Problem* (TMDSP), de Andrade et al. (2019):

Dados

- J — **tarefas**: veículos, com demanda d_j .
- M — **máquinas**: setores da rede.
- T — **períodos** do horizonte.
- C_{mt}, η_{mt} — capacidade e taxa da máquina, **variáveis no tempo**.
- q_{jmt} — capacidade que j consome em m no período t ; **só existe** se o veículo estiver sob aquele setor ali.

Regras

- Cada tarefa usa **uma máquina por período**, no máximo;
- é **preemptável entre períodos** e fica ativa até atender d_j ;
- C_j é o **último período ativo** de j .

Do FOTA ao TMDSP

O FOTA é modelado pelo *Time- and Machine-Dependent Scheduling Problem* (TMDSP), de Andrade et al. (2019):

Dados

- J — **tarefas**: veículos, com demanda d_j .
- M — **máquinas**: setores da rede.
- T — **períodos** do horizonte.
- c_{mt}, η_{mt} — capacidade e taxa da máquina, **variáveis no tempo**.
- q_{jmt} — capacidade que j consome em m no período t ; **só existe** se o veículo estiver sob aquele setor ali.

Regras

- Cada tarefa usa **uma máquina por período**, no máximo;
- é **preemptável entre períodos** e fica ativa até atender d_j ;
- C_j é o **último período ativo** de j .
- Por período: $r_{\max}$ limita a vazão total da rede e $k_{\max}$, o número de tarefas ativas.

Do FOTA ao TMDSP

O FOTA é modelado pelo *Time- and Machine-Dependent Scheduling Problem* (TMDSP), de Andrade et al. (2019):

Dados

- J — **tarefas**: veículos, com demanda d_j .
- M — **máquinas**: setores da rede.
- T — **períodos** do horizonte.
- C_{mt}, η_{mt} — capacidade e taxa da máquina, **variáveis no tempo**.
- q_{jmt} — capacidade que j consome em m no período t ; **só existe** se o veículo estiver sob aquele setor ali.

Regras

- Cada tarefa usa **uma máquina por período**, no máximo;
- é **preemptável entre períodos** e fica ativa até atender d_j ;
- C_j é o **último período ativo** de j .
- Por período: $r_{\max}$ limita a vazão total da rede e $k_{\max}$, o número de tarefas ativas.

Objetivo original, mono-objetivo: $\min \sum_{j \in J} C_j$ — o tempo total de conclusão.

Minimizar $\sum_j C_j$ não controla o *makespan*

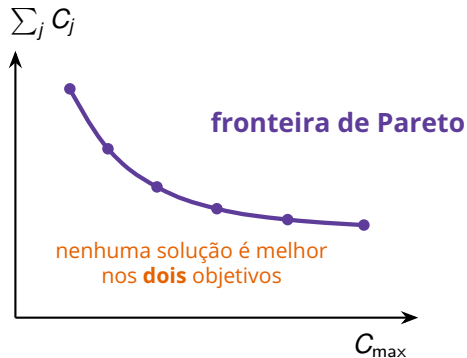
A correlação entre os dois critérios é **fraca**:
um escalonamento ótimo em $\sum_j C_j$ pode
deixar algumas tarefas terminando **muito**
depois das demais.

Propomos o **MOTMDSP**, que minimiza
simultaneamente:

$$\min C_{\max} = \max_{j \in J} C_j \quad (\textit{makespan})$$

$$\min \sum_{j \in J} C_j \quad (\text{tempo total de conclusão})$$

Observado por Andrade et al. (2019).



MOTMDSP: formulação

$$\min C_{\max} \quad \text{e} \quad \min \sum_{j \in J} C_j$$

sujeito a, para todo período t (com $\hat{t} = t \bmod |T|$):

$$\sum_{m \in M} x_{jmt} \leq 1 \quad \text{uma máquina por período, para cada tarefa } j$$

$$\sum_{j \in J} y_{jmt} \leq c_{m\hat{t}} \quad \text{capacidade da máquina } m$$

$$\sum_{j \in J} \sum_{m \in M} \eta_{m\hat{t}} y_{jmt} \leq r_{\max} \quad \text{vazão total da rede}$$

$$\sum_{j \in J} a_{jt} \leq k_{\max} \quad \text{tarefas ativas simultaneamente}$$

$$\sum_{m \in M} \sum_t \eta_{m\hat{t}} y_{jmt} = d_j \quad \text{demanda da tarefa } j \text{ atendida}$$

$$C_j \geq t a_{jt}, \quad C_{\max} \geq C_j \quad \text{tempos de conclusão}$$

x_{jmt} : atribuição; y_{jmt} : capacidade usada; a_{jt} : tarefa ativa. As restrições de início, propagação de atividade e término seguem Andrade et al. (2019).

O que vem da literatura, o que é deste trabalho

Da literatura

TMDSP e instâncias reais de FOTA

Andrade et al. (2019)

Decodificador mono-objetivo

Andrade et al. (2019)

NS-BRKGA

Mendes et al. (2022); Pauleti Mendes et al. (2023)

NSGA-II, NSPSO, MOEA/D-DE, MHACO, IHS (PaGMO2)

Deste trabalho

MOTMDSP: extensão biobjetivo. Até onde sabemos, a **primeira** formulação multiobjetivo do problema

Estendido para avaliar os dois objetivos:

$$\mathbf{F} = (C_{\max}, \sum_j C_j)$$

Primeira aplicação ao MOTMDSP, com o motor evolutivo **inalterado**

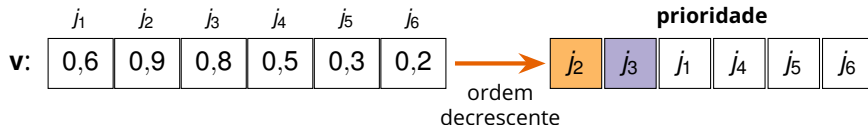
Avaliação em 96 instâncias de larga escala, com o **mesmo decodificador** para todos

Chaves aleatórias definem prioridades de escalonamento

Um cromossomo é um vetor com **uma chave por tarefa**:

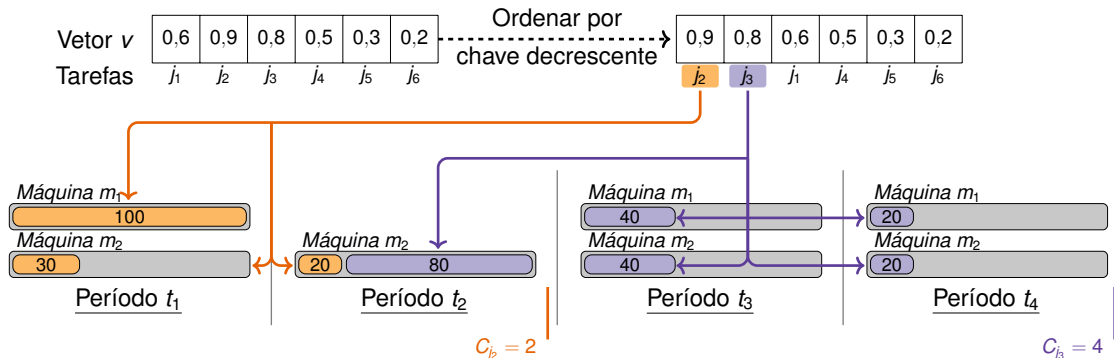
$$\mathbf{v} = (v_1, \dots, v_{|J|}), \quad v_j \in [0, 1).$$

Decodificação: ordenar as tarefas por valor **decrecente** da chave $\Rightarrow$ ordem de prioridade.



Pequenas perturbações nas chaves causam pequenas mudanças na ordem: o espaço de permutações é explorado sem operadores especializados.

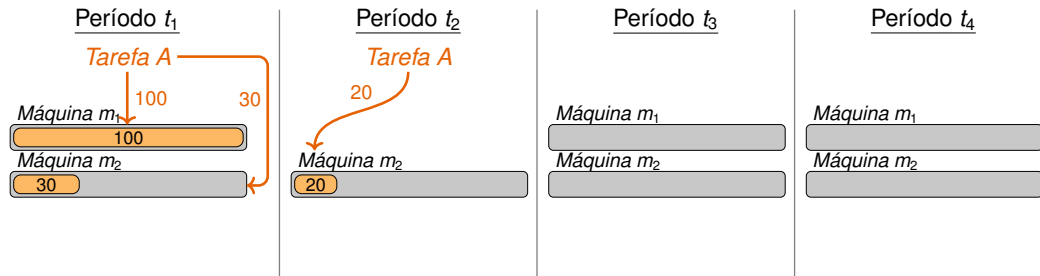
Decodificador: de prioridades a escalonamentos viáveis



Na ordem de prioridade, cada tarefa ocupa gulosamente os intervalos máquina-período disponíveis mais cedo. C_j é o último período em que a tarefa esteve ativa; alocadas *todas* as tarefas, o decodificador devolve $\mathbf{F} = (C_{\max}, \sum_j C_j)$.

As cores identificam a tarefa. Decodificador de Andrade et al. (2019), aqui estendido para devolver os **dois** objetivos

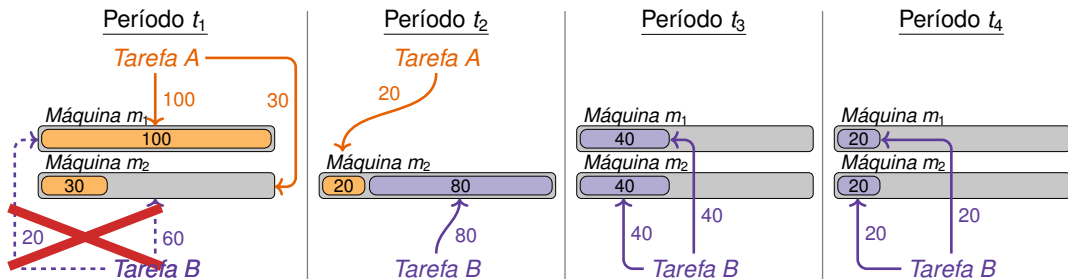
Por que uma tarefa precisa esperar



A tarefa A, de maior prioridade, consome a capacidade disponível em t_1 .

Removidas as tarefas não escalonáveis no pré-processamento, o decodificador sempre produz escalonamento viável: sem reparo nem penalidade.

Por que uma tarefa precisa esperar

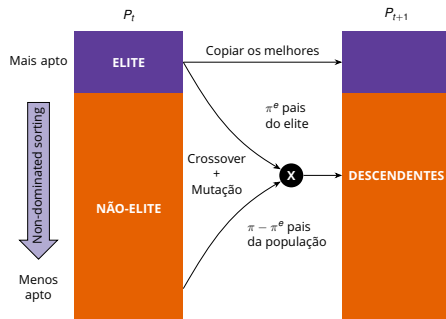


Sob c_{mt} , $r_{\max}$ e $k_{\max}$ *simultaneamente*, não resta alocação viável para a tarefa B em t_1 : ela é **empurrada** e só começa em t_2 .

Removidas as tarefas não escalonáveis no pré-processamento, o decodificador sempre produz escalonamento viável: sem reparo nem penalidade.

Ciclo evolutivo do NS-BRKGA

O motor evolutivo é **independente do problema**: toda a lógica do MOTMDSP está no decodificador.



O elite vem do **non-dominated sorting**, com **crowding distance** e diversidade genotípica no desempate. Compõem ainda o método múltiplas populações com migração, *implicit path-relinking*, *shaking* e *reset* parcial.

Configuração experimental

Instâncias	96 instâncias reais de FOTA 48 <i>common</i> + 48 <i>rare</i> até 9.706 tarefas, 9.919 máquinas 672 períodos
Algoritmos	NS-BRKGa NSGA-II, NSPSO, MOEA/D-DE, MHACO, IHS (PaGMO2)
Protocolo	5 execuções por algoritmo 900 s por execução Xeon E3-1230 V2, 32 GB, 1 núcleo

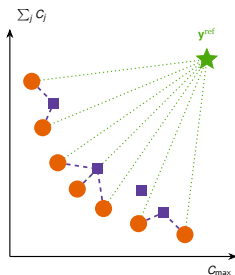
Fronteira de referência

- Uma por instância: todas as soluções não dominadas de **todos** os algoritmos e execuções.
- Filtrada por *non-dominated sorting* + *crowding distance* para até 800 pontos.
- Cada fronteira obtida é podada a até 500 pontos.

Todos os métodos usam o mesmo decodificador: muda apenas o motor de busca.

Como medimos a qualidade de uma fronteira

$NIGD^+$ ($\downarrow$): convergência



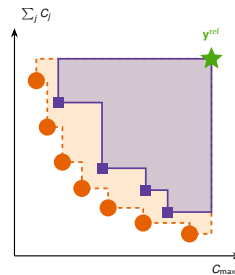
Distância média de cada ponto da referência ao obtido mais próximo, normalizada por $\mathbf{y}^{\text{ref}}$.

● fronteira de referência $\mathbf{y}^{\text{ref}}$

■ fronteira obtida S

★ pior caso $\mathbf{y}^{\text{ref}}$

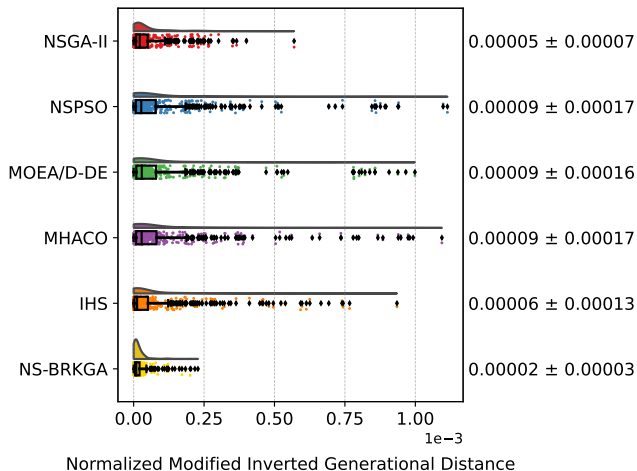
HVR ($\uparrow$): convergência + diversidade



Fração da área dominada pela referência (laranja) que a fronteira obtida (azul) cobre.

$NIGD^+$ perto de 0 é bom; HVR perto de 1 é bom. Ambos ficam em $[0, 1]$.

NS-BRKGA obtém o menor $NIGD^+$ médio global



Média de $NIGD^+$ nas 96 instâncias:

NS-BRKGA 0,00002

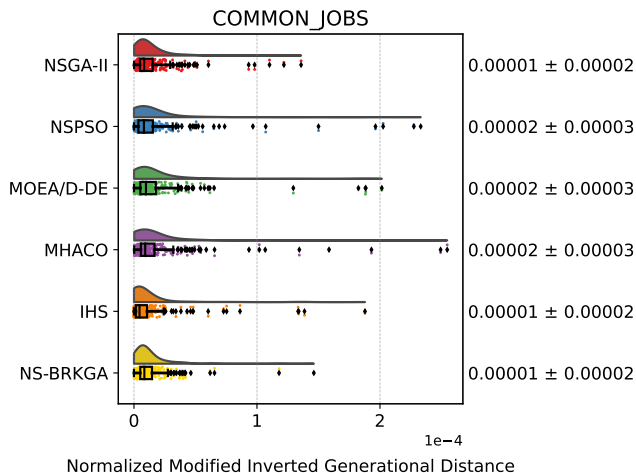
NSGA-II 0,00005

IHS 0,00006

demais 0,00009

Todos superam $HVR \geq 0,9998$, de modo que é o $NIGD^+$ que separa os métodos neste regime. O NS-BRKGA também tem a maior média de HVR e o menor desvio-padrão nos dois indicadores.

Em instâncias `common`, os seis métodos empatam

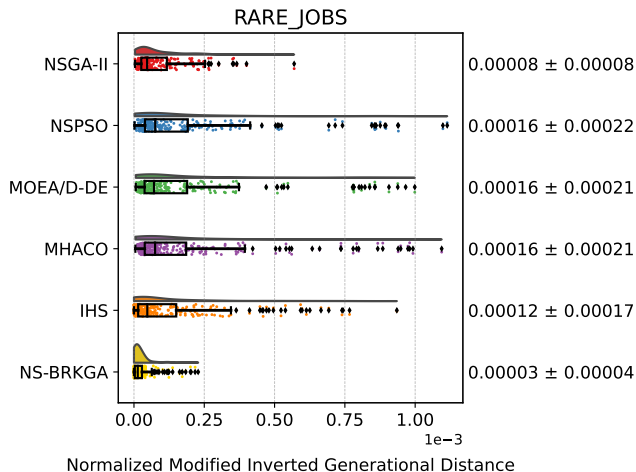


Carros que aparecem em **muitos setores** durante a janela de escalonamento.

Todas as médias de $NIGD^+$ ficam entre **0,00001 e 0,00002**: o NS-BRKA empata com NSGA-II e IHS no melhor valor.

Aqui a escolha do algoritmo **pouco importa**.

A vantagem aparece nas instâncias `rare`



Carros que aparecem em **poucos setores e períodos**. A convergência fica mais difícil para todos.

NS-BRKGa 0,00003

NSGA-II 0,00008

IHS 0,00012

demais 0,00016

Neste subconjunto o NS-BRKGa também tem o melhor *HVR* médio (0,99997).

Conclusões

- Formulamos o **MOTMDSP**, extensão biobjetivo do TMDSP que trata explicitamente o compromisso entre *makespan* e tempo total de conclusão.
- **Estendemos o decodificador** de Andrade et al. (2019) para os dois objetivos, o que permitiu aplicar ao problema tanto o NS-BRKGA quanto os cinco algoritmos de referência.
- Em 96 instâncias de larga escala, o NS-BRKGA obteve as **melhores médias** de *HVR* e *NIGD*⁺, com vantagem mais nítida nas instâncias *rare*.
- O desempenho **se mantém** à medida que o número de tarefas ou de máquinas cresce, sob o mesmo orçamento de tempo.
- **Uso prático:** em vez de um único escalonamento, o operador recebe um **conjunto de compromissos** entre terminar a campanha mais cedo e concluir mais veículos antes.

Obrigado!

Perguntas?

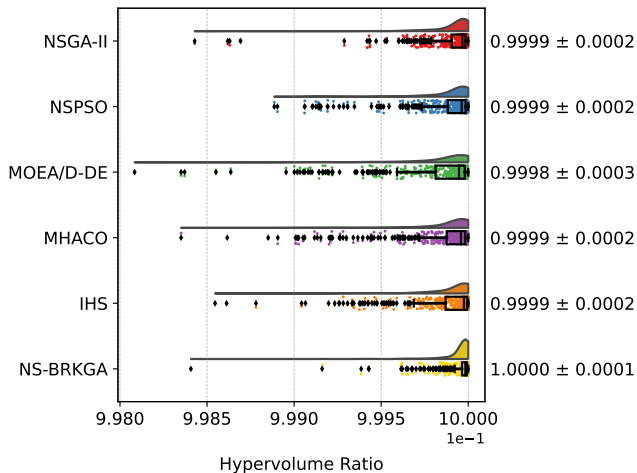
`luis.mendes@ic.unicamp.br`

`github.com/luishpmendes/ns-brkg` · `github.com/ceandrade/tmdsp_instances`

Apoio: parâmetros do NS-BRKGA

Parâmetro	Valor
Tamanho da população $ P $	300
Número de pais π (elitistas π_e)	3 (com $\pi_e = 2$)
Fração de elite $p_{\min}^e - p_{\max}^e$	0,10 – 0,30
Função de viés $\Phi(r)$	$1/\sqrt{r}$
Probabilidade de mutação p_m (η_m)	0,01 ($\eta_m = 50$)
Número de subpopulações	3
Migração	200 gerações, $k = 30$ soluções
<i>Implicit path-relinking</i>	a cada 500 gerações
<i>Shaking</i>	200 gerações sem melhora
<i>Reset parcial</i>	500 gerações sem melhora

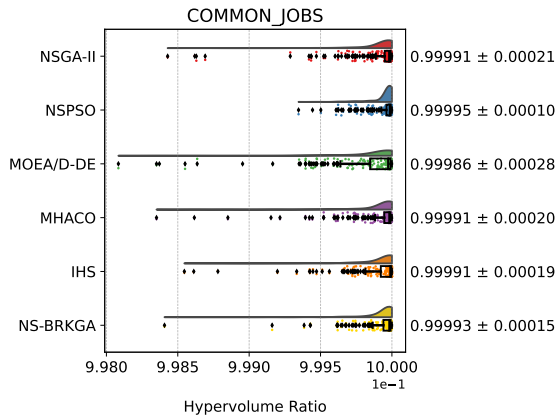
Apoio: *HVR* em todas as instâncias



Todos os algoritmos superam 0,9998 e o NS-BRKGa arredonda para 1,0000 nesta precisão.

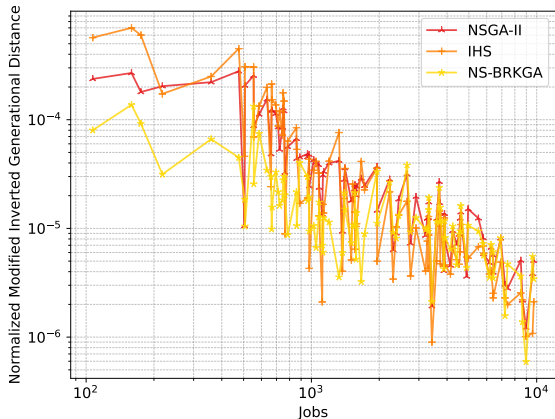
Neste regime o indicador está próximo da saturação, e é o $NIGD^+$ que dá mais resolução entre os métodos.

Apoio: *HVR* por tipo de instância

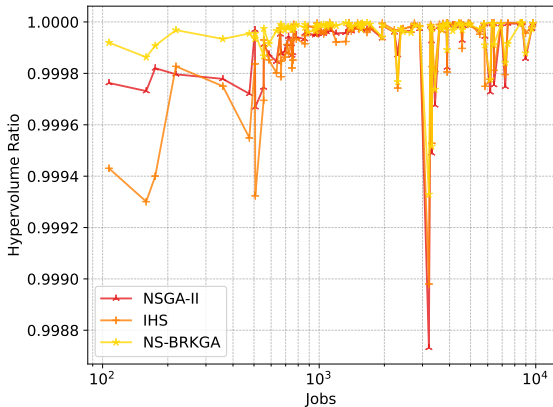


Em `common` o melhor *HVR* médio é do NSPSO (0,99995), com o NS-BRKGA em segundo (0,99993); em `rare` o NS-BRKGA lidera (0,99997).

Apoio: indicadores em função do tamanho da instância



$NIGD^+$ ($\downarrow$) vs. número de tarefas



HVR ($\uparrow$) vs. número de tarefas

Recorte com NSGA-II, IHS e NS-BRKGa, como no artigo. Sem degradação sistemática com o crescimento da instância.